Daily Quant Digest: 2026-10-26 (b)

Foundations Restart, day 1. From today all three study tracks start again from the very first brick and build up one lesson at a time (see `CURRICULUM.md`). Nothing will be used before it has been explained.

Today's topics

Track	Topic	Difficulty
Probability & Statistics	FR Lesson 1: what probability is (outcomes, sample space, events, long-run frequency)	Easy
Derivatives	FR Lesson 1: interest rates (simple, compound, continuous, and where e comes from)	Easy
Algorithms & DS	FR Lesson 1: Big-O, counting steps; the best single trade on a price series	Easy
Music	The Beatles, "A Day in the Life" (1967)	

---

1. Probability & Statistics: What Probability Is

> **Where we are.** This is Lesson 1, the ground floor. We assume nothing. By the end you should be able to say exactly what "the probability is 1/6" means, and how to compute it by counting.

1.1 The central picture: a room full of possible worlds

Before you roll a die, the future has not happened yet. Several futures are possible. Picture each possible future as a person standing in a room. Before the roll, the room holds six people, wearing badges "1", "2", "3", "4", "5", "6". When you roll, the universe picks **one** person from the room, and that is what actually happens.

We will keep this picture all through the course. Here is the dictionary between the picture and the maths:

In the picture	In the maths	Die example
The thing you do (roll, deal, toss)	An **experiment**	Roll one die
One person in the room	An **outcome**	"the die shows 4"
The whole room	The **sample space** Ω (omega)	$\{1, 2, 3, 4, 5, 6\}$
"Everyone matching a description, raise your hand"	An **event** (a set of outcomes)	"even" $= \{2, 4, 6\}$
The fraction of hands up (if everyone is equally likely to be picked)	The **probability** of the event	$3/6 = 1/2$

Three things are worth saying slowly.

An outcome is one complete answer. "The die shows 4" is an outcome. "The die shows an even number" is not an outcome, because it does not pin down what happened. It is a *description* that several outcomes match. That is an event.

An event is just a group of people in the room. We write it as a set with curly brackets: $A = \{2, 4, 6\}$. Even a single outcome can be an event ($\{4\}$, "exactly a four"). The empty group $\{\}$ is the event "nothing in the room matches", for example "the die shows 7". The whole room Ω is the event "something happens", which is certain.

The sample space must list every possible outcome, each exactly once. If you forget a person, or count someone twice, every probability you compute later is wrong.

1.2 The counting rule (when everyone is equally likely)

If the experiment is fair, so that every person in the room is equally likely to be picked, then probability is pure counting:

$$P(A) = \frac{\text{number of outcomes in } A}{\text{number of outcomes in } \Omega} = \frac{|A|}{|\Omega|}.$$

Reading it aloud: "The probability of A equals the size of A divided by the size of the whole sample space." The vertical bars $|A|$ just mean "how many things are in A".

With numbers: for "even" on one die, $|A| = 3$ and $|\Omega| = 6$, so $P(\text{even}) = 3/6 = 1/2$.

Three facts follow immediately, and you can see each one in the room:

- A probability is always between 0 and 1, because the group with hands up can be no smaller than nobody and no bigger than everybody.
- $P(\Omega) = 1$ (everyone raises a hand) and $P(\{\}) = 0$ (no one does).
- $P(\text{not } A) = 1 - P(A)$: the people with hands down are everyone else. "Not even" has $6 - 3 = 3$ people, so probability $1 - 1/2 = 1/2$.

1.3 A bigger room: two dice

Roll a red die and a blue die. What is one outcome? It is a complete answer: the red number **and** the blue number. So an outcome is a pair, like (red 3, blue 5), written $(3, 5)$. How many people are in the room? For each of the 6 red values there are 6 blue values, so $6 \times 6 = 36$. Lay them out as a grid:

red \ blue	1	2	3	4	5	6
1	2	3	4	5	6	**7**
2	3	4	5	6	**7**	8
3	4	5	6	**7**	8	9
4	5	6	**7**	8	9	10
5	6	**7**	8	9	10	11
6	**7**	8	9	10	11	12

Each cell is one person; the number in it is the total. Event "total is 7": the bold diagonal, 6 cells. So

$$P(\text{total} = 7) = \frac{6}{36} = \frac{1}{6}.$$

Total 2 needs $(1, 1)$ only: $1/36$. Total 12 needs $(6, 6)$ only: $1/36$. The totals are **not** equally likely, even though the 36 pairs are. This is the single most important habit of the whole lesson: **choose your outcomes so that they really are equally likely, and only then count.** The pairs are equally likely. The totals are not.

1.4 The other meaning: long-run frequency

Counting works when the room is fair and small. But what does "1/6" *mean* in the real world? The answer most quants use: **if you repeated the experiment many, many times, the fraction of times the event happens settles down near its probability.** In the picture: replay the night again and again, each time the universe picks a person from the room; keep a tally of how often a hand was up.

I simulated rolling two dice and counting how often the total was 7 (the exact answer is $1/6 \approx 0.1667$):

Number of rolls	Times total was 7	Fraction
36	7	0.194
360	56	0.156
3,600	590	0.164
36,000	6,014	0.167
360,000	59,725	0.166

Look at the pattern. With 36 rolls the fraction is quite far off. As the number of rolls grows, it wobbles less and less and settles near 0.1667. (Exactly *why* it settles, and how fast, is the law of large numbers, Lesson 33.) This is how a trading desk thinks about a strategy that "wins 55% of the time": not that any single trade is predictable, but that over thousands of trades the winning fraction will sit near 55%.

Where the analogy stops working. The room picture with "equally likely" people works for dice, coins and cards. Many real events (will the Fed cut rates?) have no fair room to count. There we still use probability, but the numbers come from data or judgement, not from counting. The rules you learn on dice will still apply to them.

1.5 Common mistakes

- **Counting outcomes that are not equally likely.** Toss two coins. "The number of heads is 0, 1 or 2, so each has probability 1/3." Wrong. The equally likely outcomes are HH, HT, TH, TT. One head is HT or TH: $2/4 = 1/2$. Even the famous mathematician d'Alembert is said to have made a version of this mistake.
- **Forgetting the order matters for "who is who".** $(2, 5)$ and $(5, 2)$ are different people in the two-dice room (red 2 blue 5, versus red 5 blue 2). Even if your dice are the same colour, nature still treats them as two separate dice.

1.6 Self-check

1. Toss three coins. How many outcomes are in the sample space?
2. Two dice: what is the probability that at least one shows a 6?
3. Two dice: what is the probability that the dice differ by exactly 2?

1.7 Interview question: the Duke's dice

(A warm-up with a famous story. Around 1620 gamblers in Italy noticed something odd, and Galileo was asked to explain it.)

You roll three fair dice and add them up.

a. List the ways to write 9 and 10 as a sum of three numbers from 1 to 6, ignoring order. How many are there for each?

b. A gambler says: "Since there are equally many ways, 9 and 10 must be equally likely." What is wrong with his reasoning?

c. Compute the exact probabilities of a total of 9 and a total of 10.

1.8 Solution

(a) Without caring about order:

Total 9	Total 10
1+2+6	1+3+6
1+3+5	1+4+5
1+4+4	2+2+6
2+2+5	2+3+5
2+3+4	2+4+4
3+3+3	3+3+4

Six each. This is exactly what puzzled the gamblers.

(b) These "ways" are descriptions, not equally likely outcomes. Like the totals in Section 1.3, they are groups of people of different sizes. The truly equally likely outcomes are ordered triples (first die, second die, third die). There are $6 \times 6 \times 6 = 216$ of them.

(c) Count how many ordered triples sit inside each description:

- Three **different** numbers, like 1, 2, 6: the first die can be any of the 3 numbers, then the second any of the remaining 2, then the third is forced. $3 \times 2 \times 1 = 6$ orders.
- Two the **same**, like 1, 4, 4: only the position of the odd one out matters (first, second or third). 3 orders.
- All three the same, like 3, 3, 3: only 1 order.

Total 9: $6 + 6 + 3 + 3 + 6 + 1 = 25$. Total 10: $6 + 6 + 3 + 6 + 3 + 3 = 27$. So

$$P(9) = \frac{25}{216} \approx 0.116, \qquad P(10) = \frac{27}{216} = \frac{1}{8} = 0.125.$$

The difference comes from 3+3+3: the all-same description has only one ordering, and 9 has it while 10 does not.

Interview trick. On three dice, the totals are symmetric around 10.5: a total of s and a total of $21 - s$ are equally likely (turn every die upside down; opposite faces add to 7, so the total becomes $21 - s$). So $P(10) = P(11)$ and $P(9) = P(12)$. That lets you check your work with a second count.

Sanity check. We confirmed both counts by listing all 216 triples by computer: 25 and 27. Also, the difference is about 0.009 per roll: invisible in a few games, but a gambler playing thousands of rounds would notice, which is exactly what happened. That is the long-run frequency meaning at work.

Self-check answers. (1) $2 \times 2 \times 2 = 8$. (2) Count the complement: neither die is a 6 in $5 \times 5 = 25$ cells, so $1 - 25/36 = 11/36$. (3) The pairs (1,3), (2,4), (3,5), (4,6) and their reverses: 8 cells, so $8/36 = 2/9$.

---

2. Derivatives: Interest Rates (Simple, Compound, Continuous)

> **Where we are.** Lesson 1 of the derivatives track. Every derivative price is built from two ingredients: what money is worth over time, and uncertainty. Today is the first ingredient: how money grows when it is lent.

2.1 The scene

It is Monday morning in Hong Kong. A corporate treasurer has USD 10 million that her company will not need for a year. She calls her bank. The bank's money markets desk quotes her **4.00% a year**. Simple question: how much will she have in a year? The surprising answer is: it depends on *how often the interest is paid*, and every market has its own convention. Traders who mix conventions up lose money on trades that "looked" profitable. So we learn this carefully, once.

Who is on each side? The treasurer is a **lender**: she gives up cash today for more cash later. The bank is a **borrower**: it pays interest because it can use the cash (to lend to someone else at a higher rate). Interest is the **rent on money**.

2.2 The central picture: a snowball

Picture a snowball rolling down a hill. As it rolls, it picks up snow in proportion to its size. A bigger snowball picks up more snow per turn. The snow it has just picked up is now part of the ball, so it too picks up snow on the next turn.

In the picture	In the maths
The starting snowball	The **principal** P (the amount you lend)
Snow picked up per full trip down the hill	The **interest rate** r (per year)
How many times you stop to pack the new snow into the ball	The **compounding frequency** m (times per year)
The size at the bottom	The **final amount** A

The whole lesson is: what difference does it make how often you pack the snow in?

2.3 Simple interest: the snow is kept in a bucket

With **simple interest**, the snow you pick up goes into a separate bucket. It never joins the ball, so it never earns more snow. Interest is paid only on the original principal:

$$A = P\,(1 + r\,t).$$

Symbols: P = principal, r = annual rate as a decimal (4% = 0.04), t = time in years, A = what you get back.
Read aloud: "you get your money back, plus the rate times the time, times your money."

With numbers: $P = 10{,}000{,}000$, $r = 0.04$, $t = 1$: $A = 10{,}000{,}000 \times 1.04 = 10{,}400{,}000$.

Where you will see it. Money market deposits and loans shorter than a year are quoted with simple interest, using a **day count**: the time t is "actual days divided by a fixed year length". USD uses 360 (called ACT/360), while HKD and GBP use 365. A 90-day USD deposit of 10 million at 4.00%:

$$\text{interest} = 10{,}000{,}000 \times 0.04 \times \frac{90}{360} = 100{,}000 \text{ USD}.$$

The ACT/360 convention quietly pays a bit more than the headline rate: a full 365-day year counts as $365/360 \approx 1.014$ "years".

2.4 Compound interest: the snow joins the ball

Now suppose the bank pays interest twice a year, and the treasurer leaves it in. After 6 months she earns half a year's rent: $10{,}000{,}000 \times 0.04/2 = 200{,}000$. Her balance is now 10.2 million. In the second half year she earns 2% on **10.2 million**, which is 204,000, not 200,000. The extra 4,000 is "interest on interest": the snow that joined the ball picking up snow of its own.

Each half year multiplies the balance by $(1 + 0.04/2) = 1.02$. Two half years: $1.02 \times 1.02 = 1.0404$. In general, with m payments a year for t years:

$$A = P\left(1 + \frac{r}{m}\right)^{mt}.$$

Read aloud: "each period multiplies your money by one plus the rate split into m pieces, and there are mt periods."
Why a power? Because growth multiplies. Each period you multiply by the same factor. Multiplying by the same number k times is written as a power k.

Here is the 10 million for one year at 4% with different packing frequencies:

Interest paid	m	Amount after 1 year (USD)	Interest earned
Once a year	1	10,400,000	400,000
Twice a year	2	10,404,000	404,000
Quarterly	4	10,406,040	406,040
Monthly	12	10,407,415	407,415
Daily	365	10,408,085	408,085
Continuously	∞	10,408,108	408,108

Two lessons jump out. More frequent compounding pays more. But the extra shrinks fast: going from monthly to daily adds only about 670 USD. The amounts are heading towards a ceiling. That ceiling is **continuous compounding**.

2.5 Where e comes from

Take the most extreme case: 100% interest, one year, 1 dollar. Pack the snow in n times:

n	$(1+1/n)^n$
1	2.000
2	2.250
4	2.441
12	2.613
365	2.715
1,000,000	2.71828

No matter how often you compound, you never pass about 2.71828. That number is called e:

$$e = \lim_{n \to \infty} \left(1 + \frac{1}{n}\right)^n \approx 2.71828.$$

Read aloud: "e is what one plus one over n, to the power n, gets closer and closer to as n grows without end." The word "limit" (lim) just means "the value it settles towards".

Now any rate r. Write $(1 + r/n)^n$ and call $k = n/r$ (so $n = rk$). Then

$$\left(1 + \frac{r}{n}\right)^n = \left(1 + \frac{1}{k}\right)^{kr} = \left[\left(1 + \frac{1}{k}\right)^k\right]^r \longrightarrow e^r.$$

The inner bracket is the same expression as before, so it goes to e. Over t years we do this t times, so:

$$\boxed{A = P\,e^{rt}}$$

With numbers: $10{,}000{,}000 \times e^{0.04} = 10{,}408{,}108$, matching the last row of the table. Why do quants love e^{rt}? Because it makes growth additive in the exponent: growing for 1 year then 2 years is $e^{r\cdot 1}e^{r\cdot 2} = e^{r\cdot 3}$. Almost every pricing formula you will meet (forwards, Black-Scholes) uses it.

2.6 Converting between conventions

The same deal can be described with different rates. 4% annual compounding and 3.922% continuous compounding pay **exactly the same money**. To convert, set the two growth factors equal for one year:

$$e^{r_c} = \left(1 + \frac{r_m}{m}\right)^m \quad \Longrightarrow \quad r_c = m \ln\!\left(1 + \frac{r_m}{m}\right).$$

Here ln (the natural logarithm) is the "undo" button for e: if $e^x = y$ then $x = \ln y$. **With numbers:** 4% annual: $r_c = \ln(1.04) = 0.03922$, so 3.922%. 5% annual: $\ln(1.05) = 4.879\%$.

A handy shortcut: money doubles when $e^{rt} = 2$, i.e. $rt = \ln 2 \approx 0.693$. So doubling time is about $69/(\text{rate in } \%)$ years; with annual compounding people round it to the **rule of 72**. At 6%: $72/6 = 12$ years (exact with annual compounding: 11.9).

2.7 Worked example: same headline rate, different money

Ten years, USD 10 million, 4%:

Convention	Formula	Amount (USD million)
Simple	$10 \times (1 + 0.04 \times 10)$	14.000
Annual compounding	10×1.04^{10}	14.802
Continuous	$10 \times e^{0.4}$	14.918

Over ten years the snowball effect is worth more than 0.8 million. Over one year it was tiny (8,108 between rows 1 and 6 of the first table), which is why people get lazy on short trades and careless on long ones.

When it went the other way. From 2016 to 2024 the Bank of Japan held its policy rate at minus 0.1%, and parts of Europe were negative for years. The snowball then *melts*: $e^{-0.001t}$ is below 1. Banks holding excess reserves were effectively paying to park cash, which is exactly why yen funding was so cheap and why the "yen carry trade" grew so large.

2.8 Likely interview questions

1. **Which is better for a lender: 6% compounded annually, or 5.9% compounded monthly?** Convert to the same basis: $(1 + 0.059/12)^{12} - 1 = 6.062\%$ effective. The monthly one pays more.
2. **Why do we use continuous compounding in models?** Growth factors multiply, so in the exponent they add, which makes calculus and combining periods easy. It is also the limit of every other convention, so it is a neutral common unit.
3. **You deposit 100 at 5% continuous for 3 years. How much, and how does that compare to annual?** $100e^{0.15} = 116.18$ versus $100 \times 1.05^3 = 115.76$.

---

3. Algorithms & Data Structures: Big-O and the Best Single Trade

> **Where we are.** Lesson 1 of the algorithms track. Before any clever data structures, we need a way to say "this method is fast" and "this one is slow" that does not depend on which computer you use. That is Big-O.

3.1 The central picture: a kitchen and its number of guests

Imagine two cooks preparing for a party. Cook A shakes the hand of every guest once at the door. Cook B insists that every guest shake hands with every other guest before dinner. With 10 guests, both take a few minutes. With 1,000 guests, Cook A is done in an hour; Cook B's party needs about half a million handshakes and never gets to dinner.

In the picture	In the code
Number of guests	The input size n (e.g. number of prices)
One handshake	One basic step (a comparison, an addition)
"One handshake per guest"	$O(n)$, **linear**
"Every pair of guests shakes hands"	$O(n^2)$, **quadratic**
How the total grows when guests double	What Big-O measures

Big-O does not care how fast each handshake is (that is the computer's speed). It cares about **how the count grows** when n grows. Double the guests: Cook A's work doubles; Cook B's work quadruples.

3.2 Counting steps, and what Big-O throws away

Suppose a program does $3n^2 + 5n + 20$ steps. For $n = 1{,}000$: $3{,}000{,}000 + 5{,}000 + 20$. The $3n^2$ term is 99.8% of the work. Big-O keeps only the fastest-growing term and drops the constant in front: we say the program is $O(n^2)$, read "order n squared". The constant still matters in real life (a 3x speed-up is real money in trading), but it never changes *which* method wins for large n.

The common speeds, and what they mean for $n = 10^6$ (one million prices) at roughly 10^9 simple steps per second:

Big-O	Name	Steps for $n = 10^6$	Rough time
$O(1)$	constant	1	instant
$O(\log n)$	logarithmic	20	instant
$O(n)$	linear	10^6	1 millisecond
$O(n \log n)$	"n log n"	2×10^7	20 milliseconds
$O(n^2)$	quadratic	10^{12}	about 17 minutes

Here $\log n$ means $\log_2 n$: how many times you can halve n before reaching 1. $2^{20} \approx 10^6$, so $\log_2(10^6) \approx 20$. This is why "n^2 versus $n \log n$" is the difference between a tool that works and one that does not. A useful interview rule of thumb: with n up to about 10^5 you need $O(n \log n)$ or better; $O(n^2)$ is fine up to a few thousand.

3.3 Problem: the best single trade

You are given the closing prices of a stock for n days, `prices[0..n-1]`. You may buy one share on one day and sell it on a **later** day. Return the largest profit you can make. If no trade makes money, return 0 (you just do not trade).

Constraints: $0 \le n \le 10^5$, $1 \le \text{price} \le 10^4$.

Examples:

- `[148, 146, 151, 149, 155, 150]` gives **9** (buy at 146 on day 1, sell at 155 on day 4).
- `[155, 153, 150, 149]` gives **0** (prices only fall).
- `[150]` gives **0** (no later day to sell).
- `[3, 8, 1, 6]` gives **5** (buy 3, sell 8; the later 1 is cheaper but only leaves a profit of 5 too, never more).

3.4 Brute force first

The most honest method: try every pair (buy day i, sell day j) with $i < j$, compute `prices[j] - prices[i]`, keep the best. How many pairs? For buy day 0 there are $n-1$ sell days, for day 1 there are $n-2$, and so on down to 0. Adding up:

$$(n-1) + (n-2) + \cdots + 1 + 0 = \frac{n(n-1)}{2}.$$

(Why: pair the first and last terms, $(n-1)+0$, then the second and second-last, $(n-2)+1$; each pair adds to $n-1$ and there are $n/2$ pairs.) For $n = 10^5$ that is about 5×10^9 steps: several seconds, too slow. It is Cook B shaking every pair of hands. It is $O(n^2)$.

3.5 The insight: remember the cheapest day so far

Fix a sell day j. Which buy day is best for it? Obviously the **cheapest price before day j**. So we do not need to try every buy day; we only need to *know* the cheapest one we have seen so far.

Walk through the days once, carrying two numbers in your head:

- `cheapest`: the lowest price seen up to now,
- `best`: the best profit found up to now.

At each day with price p: first ask "if I sell today, having bought at the cheapest day so far, what do I make?", i.e. $p - \text{cheapest}$, and update `best`. Then update `cheapest` in case today is the new lowest.

Trace on `[148, 146, 151, 149, 155, 150]`:

Day	Price	Profit if sold today	best	cheapest after today
0	148	0	0	148
1	146	-2	0	146
2	151	5	5	146
3	149	3	5	146
4	155	9	9	146
5	150	4	9	146

Answer 9, in 6 steps instead of 15. Each day costs a constant amount of work, so the total is $O(n)$ time and $O(1)$ extra space (two numbers, whatever n is). This is Cook A.

Why update `best` before `cheapest`? If we updated `cheapest` first, then on a new low day we would compute $p - p = 0$, which is harmless here. But the order "sell, then consider buying" matches the rule that you must buy *before* you sell, and it is a good habit for harder variants where the order really matters.

The program below counts steps for both methods. The counts are exactly $n(n-1)/2$ and n:

n	Brute force steps	One-pass steps
10	45	10
100	4,950	100
1,000	499,500	1,000
10,000	49,995,000	10,000

Each time n grows 10x, brute force grows about 100x. That is what $O(n^2)$ looks like in real numbers.

3.6 C++ solution

```cpp
#include <algorithm>
#include <cassert>
#include <iostream>
#include <random>
#include <vector>

// Brute force: try every (buy day, sell day) pair. About n*n/2 steps: O(n^2).
long long bestTradeBrute(const std::vector<long long>& prices, long long& steps) {
    long long best = 0;                       // doing nothing earns 0
    const int n = static_cast<int>(prices.size());
    for (int buy = 0; buy < n; ++buy) {
        for (int sell = buy + 1; sell < n; ++sell) {
            ++steps;
            best = std::max(best, prices[sell] - prices[buy]);
        }
    }
    return best;
}
```

```cpp
// One pass: remember the cheapest price seen so far. About n steps: O(n).
long long bestTradeFast(const std::vector<long long>& prices, long long& steps) {
    if (prices.empty()) return 0;
    long long cheapest = prices[0];           // best buy price among days seen so far
    long long best = 0;
    for (long long p : prices) {              // range-for: p is a copy of each price
        ++steps;
        best = std::max(best, p - cheapest);  // sell today, bought at the cheapest day
        cheapest = std::min(cheapest, p);     // today may become the new cheapest day
    }
    return best;
}

int main() {
    // 1) The examples from the problem statement.
    struct Case { std::vector<long long> prices; long long expected; };
    const std::vector<Case> cases = {
        {{148, 146, 151, 149, 155, 150}, 9},
        {{155, 153, 150, 149}, 0},
        {{150}, 0},
        {{3, 8, 1, 6}, 5},
    };
    for (const auto& c : cases) {
        long long s1 = 0, s2 = 0;
        long long a = bestTradeBrute(c.prices, s1);
        long long b = bestTradeFast(c.prices, s2);
        std::cout << "brute=" << a << " fast=" << b
                  << " expected=" << c.expected << '\n';
        assert(a == c.expected && b == c.expected);
    }

    // 2) Randomized cross-check: fast must equal brute force.
    std::mt19937 rng(2026);
    std::uniform_int_distribution<int> lenDist(0, 60);
    std::uniform_int_distribution<long long> priceDist(1, 1000);
    for (int trial = 0; trial < 20000; ++trial) {
        std::vector<long long> prices(lenDist(rng));
        for (auto& p : prices) p = priceDist(rng);
        long long s1 = 0, s2 = 0;
        assert(bestTradeBrute(prices, s1) == bestTradeFast(prices, s2));
    }
    std::cout << "20000 random tests passed\n";

    // 3) Count the steps as n grows.
    std::cout << "n, brute steps, fast steps\n";
    for (int n : {10, 100, 1000, 10000}) {
        std::vector<long long> prices(n);
        for (auto& p : prices) p = priceDist(rng);
        long long s1 = 0, s2 = 0;
        bestTradeBrute(prices, s1);
        bestTradeFast(prices, s2);
        std::cout << n << ", " << s1 << ", " << s2 << '\n';
    }
    return 0;
}
```

The full file (with a header comment) is `2026-10-26-b.cpp`. Compile with `g++ -std=c++17 -O2 -Wall 2026-10-26-b.cpp`. It prints all four examples correct, passes 20,000 random comparisons against brute force, and prints the step table above.

3.7 Walkthrough of the C++

The headers. `#include <vector>` gives `std::vector`, C++'s resizable array. `<algorithm>` gives `std::max` and `std::min`. `<cassert>` gives `assert`, which stops the program loudly if a condition is false: a cheap self-test. `<random>` gives the random number generator, `<iostream>` gives `std::cout` for printing.

`long long`. A 64-bit integer. Prices here fit in `int`, but on real data (prices in cents times quantities) sums overflow 32 bits quickly. Using `long long` by default for money is a good habit.

`const std::vector<long long>& prices`. Two ideas in one. The `&` makes it a **reference**: the function works on the caller's vector directly instead of copying it. Copying 10^5 numbers on every call would itself cost $O(n)$. The `const` promises the function will not change it, and the compiler enforces that promise.

`long long& steps`. Also a reference, but *not* const: the function adds to the caller's counter. This is how a function can "return" a second value. Without the `&` it would increment its own private copy and the caller would always see 0.

`static_cast<int>(prices.size())`. `size()` returns an unsigned type (`size_t`). Mixing unsigned and signed numbers in comparisons is a classic C++ bug source (and `-Wall` warns about it), so we convert once, explicitly.

The double loop. `for (int sell = buy + 1; ...)` starts one day after the buy day, which enforces "sell later than buy". The inner body runs $n(n-1)/2$ times in total: you can *see* the $O(n^2)$ in the shape of two nested loops over the same data.

`for (long long p : prices)`. A **range-based for loop**: "for each element `p` in `prices`". `p` is a copy of each element, which is fine for a small number. One loop, constant work inside: $O(n)$.

`struct Case { ... };` A tiny bundle of data (the prices and the expected answer). The list of test cases is written with braces: `{{148, 146, ...}, 9}` fills `prices` and then `expected`. This is called **brace initialization**.

`for (const auto& c : cases)`. `auto` asks the compiler to work out the type (here `Case`). `const auto&` means "look at each case by reference, without copying, and without changing it". It is the standard way to loop over a container of bigger objects.

`for (auto& p : prices) p = priceDist(rng);` Here the reference matters: `auto&` makes `p` an alias for the element itself, so assigning to `p` writes into the vector. With plain `auto` the vector would stay all zeros.

The random generator. `std::mt19937 rng(2026)` is the Mersenne Twister, seeded with 2026 so every run gives the same numbers (reproducible tests). `std::uniform_int_distribution<long long> priceDist(1, 1000)` turns its raw output into a fair whole number from 1 to 1000. You call it as `priceDist(rng)`.

`for (int n : {10, 100, 1000, 10000})` loops over a short list written inline, called an **initializer list**.

The habit to keep. Write the slow, obviously-correct version first, then the fast one, then make the computer compare them on thousands of random inputs. In an interview you mention this; in real work you do it. Every future lesson will follow the same pattern.

---

4. Music: The Beatles, "A Day in the Life" (1967)

Context

"A Day in the Life" is the closing track of *Sgt. Pepper's Lonely Hearts Club Band* (1967), the album often named as the moment rock became an art form. It is credited to Lennon and McCartney, as all their songs were, but here the joins are easy to hear: John Lennon wrote the main dreamy verses, and Paul McCartney supplied the brisk middle section about a morning rush to catch the bus, a separate unfinished song of his that fitted like a puzzle piece. George Martin produced it at Abbey Road.

The story behind it

Lennon built his verses from the newspaper. One story was the death of Tara Browne, a young Guinness heir and friend of the band, in a car crash. Another was a short item about the number of potholes counted in Blackburn, Lancashire. Lennon turned these ordinary news items into something detached and haunting, as if he were reading the paper half asleep.

The two halves needed linking. The band left 24 empty bars, with their roadie Mal Evans counting them out loud (an alarm clock he set to mark the end of the gap was left on the record, and it lands just as McCartney's "waking up" section begins). McCartney's idea for the gap was an orchestra rising from its lowest note to its highest, each player at their own pace. On 10 February 1967 a 40-piece orchestra played it, with the musicians in evening dress and party props, and the band recorded it several times and layered the takes.

The song ends with one huge E major chord struck on several pianos at once, left to ring for around 40 seconds while the engineers turned the recording level up as it faded.

The BBC refused to play it at the time, over a line it read as a drug reference.

Why people love it

It sounds like two worlds: Lennon's dream and McCartney's alarm-clock reality, held together by chaos. It is both a pop song and a piece of sound design, and it showed every band after it that a studio could be an instrument. Many critics and fans rank it as the Beatles' greatest song.

What to listen for

- **Ringo Starr’s drums.** Loose, rolling fills that answer the vocal like a second voice, nothing like a normal beat.
- **The two orchestra rises.** Listen to how the orchestra starts as a low murmur and spreads upward into a scream. No one is playing the same note at the same time.
- **The handover.** Hear the alarm clock and the sudden change in tempo and mood as McCartney’s section starts, then the “dream” sequence that carries us back to Lennon.
- **The last chord.** Use headphones and listen to the very end. As the chord fades you can hear the room, and the hiss rising as the volume is pushed up.